

CSE235 Week 8: File I/O and Exception Handling

This week, we will discuss file input/output (I/O) and handling exceptions. Reading and writing data to and from files is the primary task that most computer programs accomplish. For Python, which is often used for data processing and data science, being able to read files is especially important. We'll also discuss handling exceptions. We've discussed in the past how certain invalid input to functions can cause an exception to occur. Exceptions can also occur when attempting to read a file that does not exist or when writing to a file that we do not have the correct permissions for. Therefore, understanding exceptions is also relevant to us this week.

Exception Handling

Let's discuss three types of errors that occur in programming. First, syntax errors are errors that prevent the program from being run at all. Syntax errors are caused by writing code incorrectly. If a syntax error is present, the code will fail to run. The interpreter will report an error message indicating what it detects the problem as, and will abort running the program. The only way to resolve a syntax error is to fix the code so that it no longer has any syntax errors.

```
print("Hello World!")

# this if statement is missing the colon after
# the condition. python will give up and report
# that there is a syntax error in the code
if True
    print("Goodbye")

print("Goodbye for real this time")
```

Example Output:

```
> python3 hello.py
File "/home/user/hello.py", line 6
    if True
      ^
SyntaxError: expected ':'
```

The error message indicates the file and line number that the error occurred at. It will also display the line and a caret indicating the position on the line that the error is detected at. In this case, it expected a colon after `True`, but did not find one.

The next type of error is a logic error. Logic errors occur when you write code that is syntactically correct, but does not perform the way you want it to. An example would be an incorrect condition causing a loop to never run:

```
i = 0
# should have used less than
while i > 10:
    print(i)
    i += 1
```

While the code is technically valid, as there are no syntax errors, it does not perform as expected.

Finally, let's discuss exceptions. Exceptions, like logic errors, occur while the program is running. Unlike logic errors, exceptions occur when the error is detected during execution. In this case, an exception is raised, and if left unhandled, will cause the program to terminate. For example, dividing by zero will raise a `ZeroDivisionError` exception in Python:

```
>>> 1/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>>
```

Exceptions can be handled by using a `try-except` statement. Code that may lead to an exception is placed within the block for the `try` segment. If an exception occurs within the try block, the exception is sent to the `except` block to be handled.

```
try:
    # as soon as an exception occurs, the execution will
    # jump from the try block to the except block
    1 / 0
    print("This won't print, exception occurs first")
except:
    print("An error occurred")

print("This will print since the exception was handled")
```

An exception can be aliased to allow its description to be used or printed. An exception is aliased by putting `as name` after the `except :`

```
try:
    # as soon as an exception occurs, the execution will
    # jump from the try block to the except block
    1 / 0
    print("This won't print, exception occurs first")
# aliasing the exception as ex
except as ex:
    print(ex)

print("This will print since the exception was handled")
```

The `open()` Function

The primary way to read a file in Python is using the `open()` function. `open()` expects a file path as an argument. It will open the file and return a file handle object. File has a second argument, `mode`, which specifies whether the file is being opened for reading, writing, or appending. It has a default argument of read. Below are the different options:

Mode Argument	Description
'r'	Read (default)
'w'	Write (will overwrite existing file)
'a'	Append (will add to existing file)
'x'	Create and write (fails if file exists)

Mode Argument	Description
'b'	Read file as binary
't'	Read file as text (default)
'+'	Read and write

The modes can be combined. For example, 'rb' specifies that the file is opened for reading, and will interpret the contents of the file as text. The default is 'rt', which opens the file for reading, and decodes the file's contents as text. The '+' option is never used on its own. 'r+' indicates that the file is opened for reading, but can be written to in an arbitrary location. This is different from 'a', since the 'a' mode only supports writing to the end of the file. 'a+' opens the file for appending, but allows its existing contents to be read. 'w+' truncates the file, i.e., deletes its contents, but allows the contents that are subsequently written to be read back from the file.

The next argument is `buffering`. The `buffering` argument specifies how large of an input buffer is used when reading the file. Basically, this specifies how many characters can be read at a time. The default is to not use a buffer and read the entire file.

Next up is `encoding`. This argument is only useful if the file is being read in text mode. The default is typically `utf-8` character encoding. This tells it to convert the individual bytes of the file to text characters based on the UTF-8 encoding scheme. Any encoding scheme that Python recognizes can be passed. For this class, we can assume that UTF-8 is the correct encoding.

The next argument we'll discuss is `error`. This argument is used to indicate what action is taken when an encoding/decoding error takes place. Like `encoding`, this argument is only used when the text mode is being used. Below is a table showing the different error handlers:

Error Argument	Description
strict	Raises a <code>ValueError</code> if a character cannot be decoded

Error Argument	Description
ignore	Skips characters that cannot be decoded
replace	Replaces character that cannot be decoded with a placeholder (often '?')
surrogateescape	Replaces characters that cannot be decoded with a code that can be used to recover the original character later
xmlcharrefreplace	Only used for writing. Replaces unknown characters with their XML character reference
backslashreplace	Replaces characters with Python's backslash escaped character
namereplace	Only used for writing. Replaces unknown characters with <code>N{...}</code>

The final argument is `newline`. The `newline` argument specifies which character(s) to treat as the end of a line. The default is system dependent. Mac and Linux use LF (line-feed), the `'\n'` character, for a new line in files. Windows typically uses CRLF (carriage return line-feed), the two characters `"\r\n"`, to end lines in files. There may be some applications where the carriage return `'\r'` is used as the line end character. Specifying this argument tells `open()` which character (or pair of characters) to convert to `'\n'` for processing when reading the file, or which character(s) to replace `'\n'` with when writing the file.

The `open()` function can be called to create a file object:

```
f = open("file.txt", encoding="utf-8")
```

Files that are opened must always be closed. Even if the Python script completes successfully, not closing a file may result in a corruption of the file if in a write mode:

```
f.close()
```

Files are not typically opened like this in Python, however. There is a large amount of boilerplate exception handling code that needs to be written to safely open files and ensure they are closed correctly. Python offers a shorthand to this via the `with` statement:

```
# create a file object by calling open(), and name it f
with open("file.txt", encoding="utf-8") as f:
    # print file contents
    print(f.read())

# with automatically closes the file once its block ends
print("The file is now closed")
```

The advantage of `with` statements is that they handle closing the file, even when an exception occurs. If you wanted to manually open and close the file, then the open and close would need to occur within a `try-except` statement:

```
try:
    f = open("file.txt", encoding="utf-8")
    print(f.read())
    f.close()
except:
    print("Something went wrong!")
    f.close()

print("The file is now closed")
```

File Objects

The `open()` function returns a file object, which supports operations for reading or writing to files, depending on which mode the file was opened with. The following functions are available for reading files:

Name	Description
<code>read(count)</code>	Returns a string with the whole file's content (if no argument is provided) or <code>count</code> characters from the file
<code>readline()</code>	Reads the next line from the file into a string

Name	Description
------	-------------

<code>readlines()</code>	Reads all the lines of the file into a list of strings
--------------------------	--

`read()` reads all the content in a file if it is not passed an argument. Otherwise, it can be passed an integer, which specifies the number of characters to be read. The next time `read()` is called, it will start reading from the last place it read from. `readline()` reads a single line from the file. A line is determined by the presence of the new line character. Like `read()`, `readline()` will read the next available line the next time it is called. `readlines()` reads every line of the file into a list of strings. This is equivalent to calling `read()` on the whole file, then splitting the resulting string on the new line character. Another method for reading a file is using a for loop. An iteration of a file is equivalent to reading a line. In other words, each time a for loop over a file runs, `readline()` is called. Assume we have the following text file:

```
First line of text
Second line of text.
Third line of text.
```

Assume we have a file object `f` that has opened this file for reading. Then:

```
contents = f.read()
```

will read all the content into the variable `contents`

```
text1 = f.read(10)
text2 = f.read(10)
```

will read "First line" into `text1` and " of text\nS" into `text2`.

```
line1 = f.readline()
line2 = f.readline()
```

will read "First line of text" into `line1` and "Second line of text." into `line2`.

```
lines = f.readlines()
assert len(lines) == 3
assert lines[0] == "First line of text"
assert lines[1] == "Second line of text."
assert lines[2] == "Third line of text."
```

will create a list with three elements, one for each line of the file.

```
for line in f:
    print("for loop iteration")
    print(line)
```

will output:

```
for loop iteration
First line of text
for loop iteration
Second line of text.
for loop iteration
Third line of text.
```

Here is an example of using `open()` to read a file. In this case, we'll read a CSV file. CSV is a file format that represents data as comma separated values. Each line represents a row of data. Let's say we have this CSV data:

```
Title,Author,Genre
It,Stephen King,Horror
The Hunger Games,Suzanne Collins,Young Adult
```

In this example, the first row of a CSV file is a header that describes what each item is. This file is storing information about books, including their title, author, and genre. Let's read this file using Python:

```
# assume the above csv data is in a file called books.csv
with open("books.csv") as f:
    # read the header
    header = f.readline()
    headers = header.split(",")
    # we can either use readlines() to read all lines
    # or use a for loop to read one line at a time
    # using a for loop is more memory efficient
    for line in f:
        # split on commas
```



```
parts = line.split(",")
# print the data
print(f"{headers[0]}: {parts[0]}")
print(f"{headers[1]}: {parts[1]}")
print(f"{headers[2]}: {parts[2]}")
```

Output:

```
Title: It
Author: Stephen King
Genre: Horror

Title: The Hunger Games
Author: Suzanne Collins
Genre: Young Adult
```

The `write()` function is used to write data to a file. A file must be opened for writing to be able to use this function.

```
content = """
Here is some content that we want to write to a file.
All of this content, including any escape sequences\n
will be written to the file. Since this is a multi-line
string literal, the new lines in this string are will be
written as well.
"""

with open("file.txt", mode="w") as f:
    f.write(content)
```

The `tell()` member function gives the current character position of the file pointer. When reading/writing to a file, it is typically done from the start of the file to the end. When opening the file in a "+" mode, where both reading and writing is supported, you may need to move around within the file. The `tell()` function gives the current position in the file. The `seek()` function is used to move to a different location in the file. Seek has two arguments. The first argument is the offset that you are seeking to. The second argument is called `whence`, and is the location that the offset is added to. `whence` supports three values. 0 indicates the beginning of the file. 1 indicates the current location (the location returned by `tell()`). 2 indicates to seek from the end of the file. Typically, files are read and written to sequentially, so `tell()` and `seek()` do not see much use.

Conclusion

Reading and writing files is an important task in programs. The `open()` function is used to open files. The `mode` argument tells open whether the function is opened for reading, writing, or appending. By default, Python interprets the contents of files as text. The mode can be changed to interpret files as binary data. For this class, we'll only consider text files. Content can be read from files using `read()`, `readline()`, `readlines()`, or a for loop. Data is written using the `write()` function.